

BluSmart

BluSmart Machine Learning Project

Project By : PRASAD JADHAV

```
In [196... import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [197... import warnings
warnings.filterwarnings('ignore')
```

```
In [198... df = pd.read_csv('blusmart_dataset.csv')
pd.set_option('display.max_columns',25)
print(df.shape)
```

(10000, 20)

```
In [4]: df.head()
```

```
Out[4]:
```

	ride_id	customer_id	driver_id	vehicle_id	pickup_location	dropoff_location	ride_time	ride
0	1	4174	309	94	Pune	Bangalore	2024-03-16 19:44:44.405147	
1	2	4507	840	176	kolkata	Delhi	2024-03-08 06:00:44.405175	
2	3	1860	961	6	Pune	Chennai	2023-11-26 16:59:44.405196	
3	4	2294	374	15	Hyderabad	Mumbai	2024-01-06 07:21:44.405203	
4	5	2130	378	143	Bangalore	Jaipur	2024-01-30 05:01:44.405214	

```
In [5]: df.tail()
```

Out[5]:	ride_id	customer_id	driver_id	vehicle_id	pickup_location	dropoff_location	ride_time	ri
	9995	9996	1524	473	51	Delhi	Pune	2024-09-13 19:22:44.432343
	9996	9997	4879	904	154	Kolkata	Chennai	2024-01-31 02:17:44.432347
	9997	9998	2624	898	70	Pune	Mumbai	2024-02-06 21:43:44.432351
	9998	9999	4907	303	20	Mumbai	Pune	2023-12-08 12:32:44.432354
	9999	10000	4113	710	23	Mumbai	Jaipur	2024-06-07 16:37:44.432356

Dataset Overview

- ride_id (int): Unique identifier for each ride
- customer_id (int): Unique identifier for each customer
- driver_id (int): Unique identifier for each driver
- vehicle_id (int): Unique identifier for each vehicle
- pickup_location (string): Location where the ride begins
- dropoff_location (string): Location where the ride ends
- ride_time (datetime): Timestamp when the ride begins
- ride_duration (int): Duration of the ride (in minutes)
- ride_distance (float): Distance of the ride (in kilometers)
- fare_amount (float): Fare charged for the ride
- payment_method (string): Payment method used (credit, debit, wallet, etc.)
- driver_rating (float): Customer rating for the driver (out of 5)
- vehicle_model (string): Model of the vehicle (e.g., Mahindra e-Verito, Tata e-Tigor)
- battery_level (float): Battery level of the vehicle at the beginning of the ride (percentage)
- charging_station_proximity (float): Distance to the nearest charging station (in kilometers)
- traffic_conditions (string): Traffic conditions during the ride (light, moderate, heavy)
- weather_conditions (string): Weather conditions during the ride (sunny, rainy, cloudy)
- cancellation_flag (bool): Indicates if the ride was canceled (True/False)
- schedule_time (datetime): Time the ride was scheduled (if applicable)
- vehicle_maintenance_flag (bool): Indicates if the vehicle requires maintenance after the ride (True/False)

Data Exploration

```
In [6]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ride_id                               10000 non-null  int64
1   customer_id                           10000 non-null  int64
2   driver_id                             10000 non-null  int64
3   vehicle_id                            10000 non-null  int64
4   pickup_location                       10000 non-null  object
5   dropoff_location                      10000 non-null  object
6   ride_time                             10000 non-null  object
7   ride_duration                         10000 non-null  int64
8   ride_distance                         10000 non-null  float64
9   fare_amount                           10000 non-null  float64
10  payment_method                        10000 non-null  object
11  driver_rating                         10000 non-null  float64
12  vehicle_model                         10000 non-null  object
13  battery_level                         10000 non-null  float64
14  charging_station_proximity            10000 non-null  float64
15  traffic_conditions                    10000 non-null  object
16  weather_conditions                    10000 non-null  object
17  cancellation_flag                     10000 non-null  bool
18  schedule_time                         8027 non-null   object
19  vehicle_maintenance_flag              10000 non-null  bool
dtypes: bool(2), float64(5), int64(5), object(8)
memory usage: 1.4+ MB

```

```
In [7]: df.isnull().sum()
```

```

Out[7]: ride_id                0
customer_id                  0
driver_id                    0
vehicle_id                   0
pickup_location              0
dropoff_location             0
ride_time                    0
ride_duration                0
ride_distance                0
fare_amount                  0
payment_method               0
driver_rating                0
vehicle_model                0
battery_level                0
charging_station_proximity    0
traffic_conditions            0
weather_conditions            0
cancellation_flag            0
schedule_time                1973
vehicle_maintenance_flag      0
dtype: int64

```

```
In [15]: df = df.dropna()
```

```
In [16]: df['schedule_time'].isna().sum()
```

Out[16]: 0

```
In [11]: df.duplicated().sum()
```

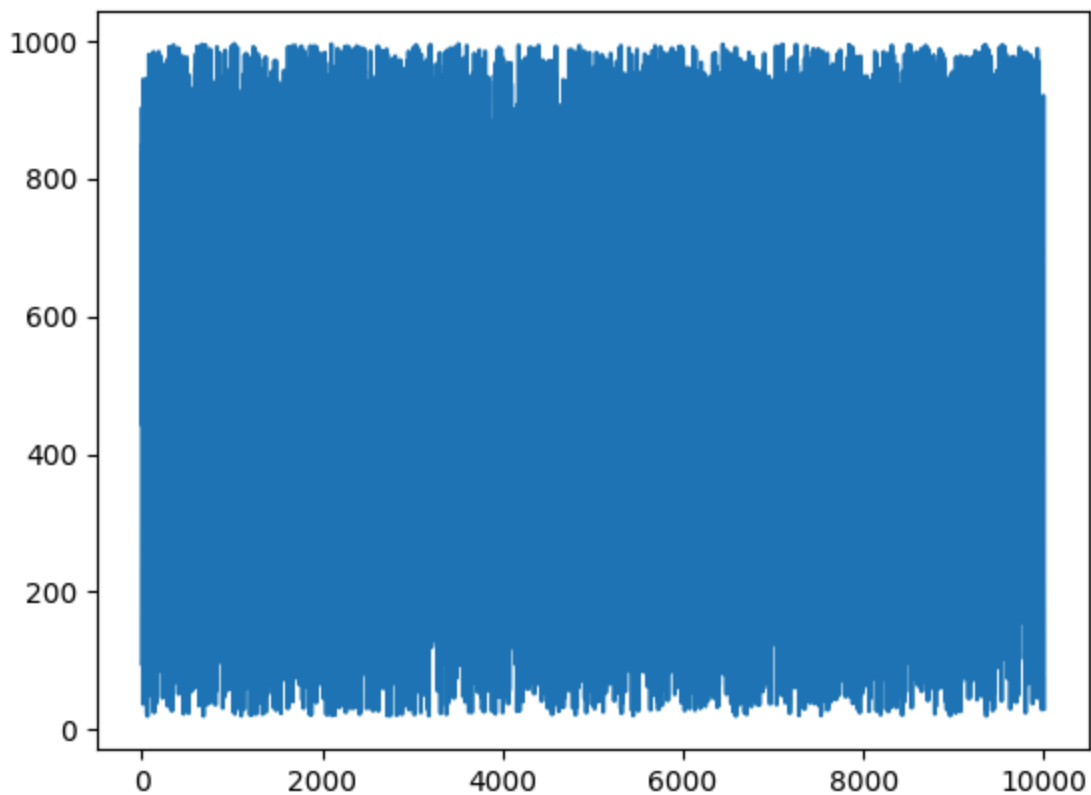
Out[11]: 0

```
In [12]: df.describe()
```

Out[12]:

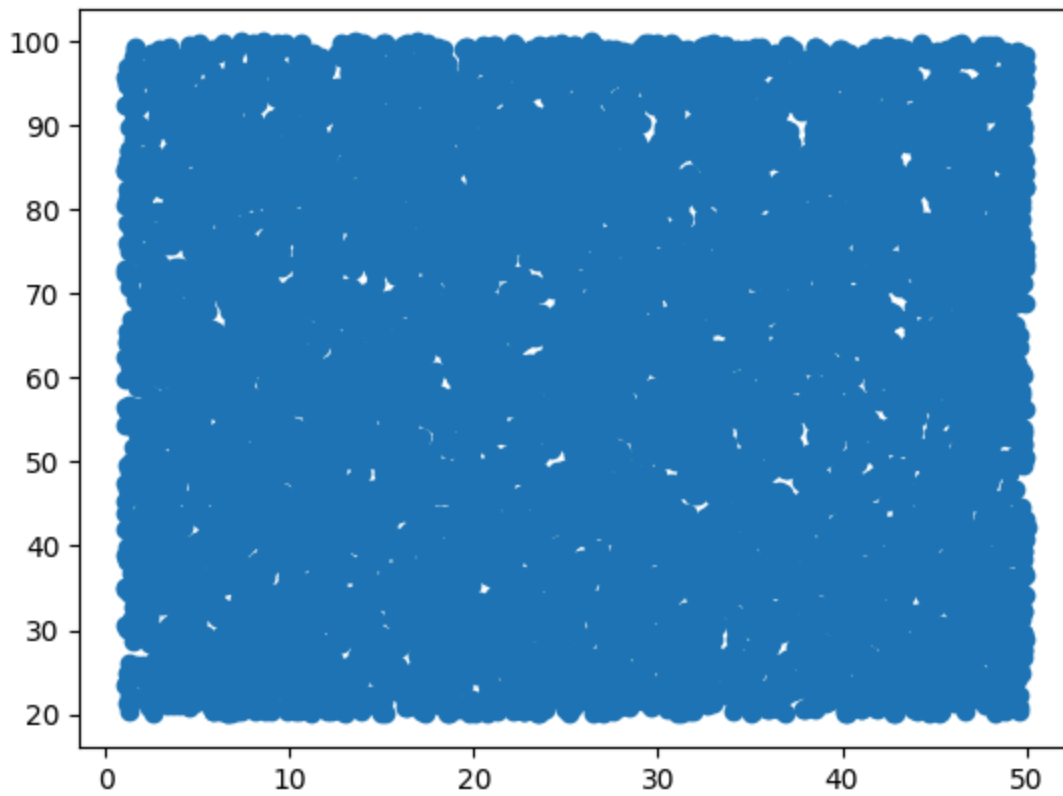
	ride_id	customer_id	driver_id	vehicle_id	ride_duration	ride_distance	fare_amount
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	3010.561100	554.392400	100.182200	61.818800	25.415050	505.683263
std	2886.89568	1152.371939	261.461055	57.473947	33.056812	14.168922	281.919110
min	1.00000	1000.000000	100.000000	1.000000	5.000000	1.010000	20.100000
25%	2500.75000	2001.750000	328.000000	50.750000	33.000000	13.280000	264.230000
50%	5000.50000	3004.000000	560.000000	100.000000	62.000000	25.300000	503.390000
75%	7500.25000	4010.000000	782.000000	150.000000	90.000000	37.760000	751.310000
max	10000.00000	4999.000000	999.000000	199.000000	119.000000	50.000000	994.850000

```
In [13]: df['fare_amount'].plot(kind='line')
plt.show()
```



```
In [14]: plt.scatter(df['ride_distance'], df['battery_level'])
```

```
plt.show()
```



```
In [17]: num_cols = [x for x in df.columns if df[x].dtypes != 'float64']

for col in num_cols:
    print(f"Value counts for column '{col}':")
    print(df[col].value_counts())
    print("\n" + "_"*40 + "\n")
```

```
Value counts for column 'ride_id':
ride_id
1      1
6631   1
6644   1
6643   1
6642   1
..
3297   1
3296   1
3295   1
3294   1
10000  1
Name: count, Length: 8027, dtype: int64
```

```
Value counts for column 'customer_id':
customer_id
3773    9
4854    8
3708    8
2462    8
1661    8
..
1612    1
4071    1
1258    1
3614    1
2624    1
Name: count, Length: 3452, dtype: int64
```

```
Value counts for column 'driver_id':
driver_id
563    22
705    19
354    18
659    18
320    18
..
550     2
487     2
729     2
197     2
928     2
Name: count, Length: 900, dtype: int64
```

```
Value counts for column 'vehicle_id':
vehicle_id
143    58
112    57
10     55
```

```
19      54
138     53
..
175     28
99      27
35      27
11      26
183     25
Name: count, Length: 199, dtype: int64
```

```
Value counts for column 'pickup_location':
pickup_location
Chennai      1069
Hyderabad    1053
Jaipur       1016
Kolkata      1012
Mumbai       981
Pune         980
Delhi        970
Bangalore    946
Name: count, dtype: int64
```

```
Value counts for column 'dropoff_location':
dropoff_location
Chennai      1086
Bangalore    1040
Jaipur       1011
Kolkata      1007
Delhi        991
Pune         985
Hyderabad    954
Mumbai       953
Name: count, dtype: int64
```

```
Value counts for column 'ride_time':
ride_time
2024-03-16 19:44:44.405147    1
2024-05-01 17:47:44.424379    1
2024-04-14 11:40:44.424405    1
2023-12-06 21:04:44.424402    1
2024-05-10 06:10:44.424401    1
..
2023-11-13 13:00:44.416352    1
2024-01-11 01:33:44.416349    1
2024-08-06 18:20:44.416346    1
2024-07-19 11:15:44.416343    1
2024-06-07 16:37:44.432356    1
Name: count, Length: 8027, dtype: int64
```

Value counts for column 'ride_duration':

ride_duration

106 94

98 93

56 89

62 88

116 88

..

78 56

118 56

23 56

92 54

105 51

Name: count, Length: 115, dtype: int64

Value counts for column 'payment_method':

payment_method

credit 1625

wallet 1616

UPI 1602

debit 1595

cash 1589

Name: count, dtype: int64

Value counts for column 'vehicle_model':

vehicle_model

Tata e-Tigor 1633

MG ZS Electric 1628

Mahindra e-Verito 1607

Hyundai Kona Electric 1580

Tata Xpres-T EV 1579

Name: count, dtype: int64

Value counts for column 'traffic_conditions':

traffic_conditions

heavy 2727

light 2671

moderate 2629

Name: count, dtype: int64

Value counts for column 'weather_conditions':

weather_conditions

rainy 2684

cloudy 2672

sunny 2671

Name: count, dtype: int64

```
Value counts for column 'cancellation_flag':
cancellation_flag
False      7235
True        792
Name: count, dtype: int64
```

```
Value counts for column 'schedule_time':
schedule_time
2024-03-16 19:33:44.405147    1
2024-05-01 17:08:44.424379    1
2024-04-14 11:29:44.424405    1
2023-12-06 20:33:44.424402    1
2024-05-10 05:46:44.424401    1
..
2023-11-13 12:29:44.416352    1
2024-01-11 01:14:44.416349    1
2024-08-06 17:55:44.416346    1
2024-07-19 10:42:44.416343    1
2024-06-07 16:06:44.432356    1
Name: count, Length: 8027, dtype: int64
```

```
Value counts for column 'vehicle_maintenance_flag':
vehicle_maintenance_flag
False      7615
True        412
Name: count, dtype: int64
```

```
In [18]: cat_cols = [col for col in df.columns if df[col].dtype == 'object' or df[col]
for col in cat_cols:
    print(f"Value counts for column '{col}':")
    print(df[col].value_counts())
    print("\n" + "_"*40 + "\n")
```

```
Value counts for column 'pickup_location':
pickup_location
Chennai      1069
Hyderabad    1053
Jaipur       1016
Kolkata      1012
Mumbai       981
Pune         980
Delhi        970
Bangalore    946
Name: count, dtype: int64
```

```
Value counts for column 'dropoff_location':
dropoff_location
Chennai      1086
Bangalore    1040
Jaipur       1011
Kolkata      1007
Delhi        991
Pune         985
Hyderabad    954
Mumbai       953
Name: count, dtype: int64
```

```
Value counts for column 'ride_time':
ride_time
2024-03-16 19:44:44.405147    1
2024-05-01 17:47:44.424379    1
2024-04-14 11:40:44.424405    1
2023-12-06 21:04:44.424402    1
2024-05-10 06:10:44.424401    1
..
2023-11-13 13:00:44.416352    1
2024-01-11 01:33:44.416349    1
2024-08-06 18:20:44.416346    1
2024-07-19 11:15:44.416343    1
2024-06-07 16:37:44.432356    1
Name: count, Length: 8027, dtype: int64
```

```
Value counts for column 'payment_method':
payment_method
credit      1625
wallet      1616
UPI         1602
debit       1595
cash        1589
Name: count, dtype: int64
```

```
Value counts for column 'vehicle_model':
vehicle_model
Tata e-Tigor          1633
MG ZS Electric        1628
Mahindra e-Verito     1607
Hyundai Kona Electric 1580
Tata Xpres-T EV       1579
Name: count, dtype: int64
```

```
Value counts for column 'traffic_conditions':
traffic_conditions
heavy      2727
light      2671
moderate   2629
Name: count, dtype: int64
```

```
Value counts for column 'weather_conditions':
weather_conditions
rainy      2684
cloudy     2672
sunny      2671
Name: count, dtype: int64
```

```
Value counts for column 'schedule_time':
schedule_time
2024-03-16 19:33:44.405147    1
2024-05-01 17:08:44.424379    1
2024-04-14 11:29:44.424405    1
2023-12-06 20:33:44.424402    1
2024-05-10 05:46:44.424401    1
..
2023-11-13 12:29:44.416352    1
2024-01-11 01:14:44.416349    1
2024-08-06 17:55:44.416346    1
2024-07-19 10:42:44.416343    1
2024-06-07 16:06:44.432356    1
Name: count, Length: 8027, dtype: int64
```

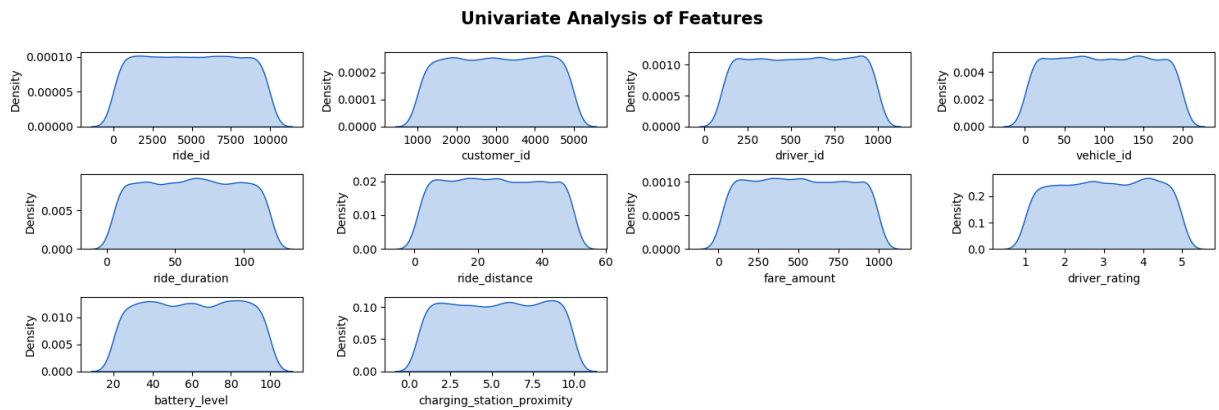
EDA

```
In [19]: num_features = df.select_dtypes(include = ['int64', 'float64']).dtypes.index
```

```
In [20]: plt.figure(figsize=(15,15))
plt.suptitle('Univariate Analysis of Features',fontweight='bold',fontsize=15

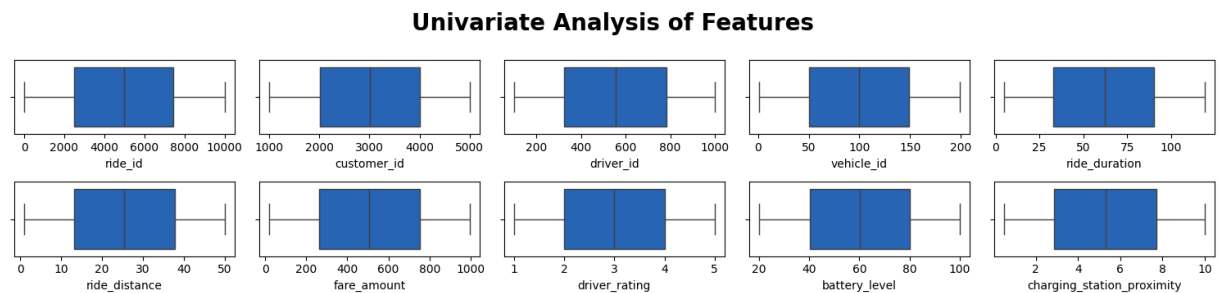
for i in range(0,len(num_features)):
```

```
plt.subplot(10,4,i+1)
sns.kdeplot(x=df[num_features[i]],shade=True,color='#1260CC')
plt.tight_layout()
```



```
In [21]: plt.figure(figsize = (15,15))
plt.suptitle('Univariate Analysis of Features',fontweight='bold',fontsize=20)

for i in range(0,len(num_features)):
    plt.subplot(10,5,i+1)
    sns.boxplot(data=df,x=num_features[i],color='#1260CC')
    plt.xlabel(num_features[i])
    plt.tight_layout()
```



```
In [ ]: cat_features = df.select_dtypes(include='object').dtypes.index
```

```
In [ ]: plt.figure(figsize=(15,15))
plt.suptitle('Univariate Analysis of Features',fontweight='bold',fontsize=15)

for i in range(0,len(cat_features)):
    plt.subplot(10,4,i+1)
    sns.countplot(x=df[cat_features[i]],color='#1260CC')
    plt.tight_layout()
```

Machine Learning Problems & Approaches

Ride Demand Forecasting:

- **Problem**: Predict the demand for rides in different areas.
- **Approach**: Time series forecasting using features like ride-time, pickup-location, ride-distance, and ride-duration.
- **Model**: ARIMA, LSTM, or Prophet.

```
In [ ]: '''
from fbprophet import Prophet

# Preprocessing for time series
df['ride_time'] = pd.to_datetime(df['ride_time'])
df['date'] = df['ride_time'].dt.date

# Aggregate ride counts per day
ride_counts = df.groupby('date').size().reset_index(name='ride_count')

# Prepare data for Prophet
prophet_df = ride_counts.rename(columns={'date': 'ds', 'ride_count': 'y'})

# Train the Prophet model
model = Prophet()
model.fit(prophet_df)

# Create a future dataframe
future = model.make_future_dataframe(periods=30)

# Forecast demand for the next 30 days
forecast = model.predict(future)

# Plot the forecast
model.plot(forecast)
'''
```

```
In [23]: # Using NeuralProphet
from neuralprophet import NeuralProphet
```

Importing plotly failed. Interactive plots will not work.
 Importing plotly failed. Interactive plots will not work.

```
In [24]: # Prepare the data
df = pd.DataFrame({
    'ds': pd.to_datetime(df['ride_time']),
    'y': df['ride_duration']
})
```

```
In [25]: # Initialize NeuralProphet model
model = NeuralProphet(yearly_seasonality=True, weekly_seasonality=True, dail
```

```
In [26]: # Fit the model
model.fit(df, freq='H') # Assuming hourly data, adjust as necessary
```

```
WARNING - (NP.forecaster.fit) - When Global modeling with local normalization, metrics are displayed in normalized scale.
INFO - (NP.df_utils._infer_frequency) - Major frequency -15159359999997us corresponds to 0.025% of the data.
WARNING - (NP.df_utils._infer_frequency) - Dataframe has multiple frequencies. It will be resampled according to given frequency H. Ignore message if actual frequency is any of the following: SM, BM, CBM, SMS, BMS, CBMS, BQ, BQS, BA, or, BAS.
INFO - (NP.config.init_data_params) - Setting normalization to global as only one dataframe provided for training.
INFO - (NP.config.set_auto_batch_epoch) - Auto-set batch_size to 64
INFO - (NP.config.set_auto_batch_epoch) - Auto-set epochs to 60
Training: |
| 0/? [00:00<...

WARNING - (NP.config.set_lr_finder_args) - Learning rate finder: The number of batches (126) is too small than the required number for the learning rate finder (247). The results might not be optimal.
Finding best initial lr: 0%| | 0/247 [00:00<?, ?it/s]
Training: |
| 0/? [00:00<...
```

Out[26]:

	train_loss	reg_loss	MAE	RMSE	Loss	RegLoss	epoch
0	0.403231	0.0	73.666763	92.280930	0.402135	0.0	0
1	0.132336	0.0	32.843636	39.415619	0.132410	0.0	1
2	0.110968	0.0	29.495907	34.617184	0.111032	0.0	2
3	0.111344	0.0	29.441704	34.477451	0.111254	0.0	3
4	0.110935	0.0	29.450212	34.492493	0.111030	0.0	4
5	0.111302	0.0	29.444838	34.442478	0.111423	0.0	5
6	0.110544	0.0	29.333235	34.261284	0.110577	0.0	6
7	0.112190	0.0	29.487625	34.525581	0.112211	0.0	7
8	0.113395	0.0	29.644613	34.829891	0.113405	0.0	8
9	0.114190	0.0	29.863489	34.990929	0.114210	0.0	9
10	0.115488	0.0	29.979477	35.312004	0.115449	0.0	10
11	0.114746	0.0	29.843630	35.152245	0.114592	0.0	11
12	0.116323	0.0	30.141226	35.549004	0.116390	0.0	12
13	0.117360	0.0	30.305393	35.713200	0.117569	0.0	13
14	0.117370	0.0	30.258823	35.738434	0.117444	0.0	14
15	0.116478	0.0	30.035490	35.592434	0.116448	0.0	15
16	0.117618	0.0	30.195168	35.857796	0.117670	0.0	16
17	0.118045	0.0	30.345116	35.973663	0.117918	0.0	17
18	0.119818	0.0	30.645292	36.281090	0.120065	0.0	18
19	0.120329	0.0	30.650229	36.342987	0.120308	0.0	19
20	0.118631	0.0	30.394119	36.055382	0.118783	0.0	20
21	0.118041	0.0	30.276703	35.906006	0.118072	0.0	21
22	0.117045	0.0	30.245867	35.598690	0.116964	0.0	22
23	0.116160	0.0	30.044201	35.548302	0.116233	0.0	23
24	0.116723	0.0	30.131348	35.542843	0.116678	0.0	24
25	0.113907	0.0	29.800608	34.881050	0.113767	0.0	25
26	0.113959	0.0	29.755470	34.896721	0.114070	0.0	26
27	0.112382	0.0	29.490326	34.541225	0.112418	0.0	27
28	0.112181	0.0	29.497284	34.504608	0.112082	0.0	28
29	0.110167	0.0	29.131977	33.979767	0.109958	0.0	29

	train_loss	reg_loss	MAE	RMSE	Loss	RegLoss	epoch
30	0.109689	0.0	29.086601	34.068657	0.109715	0.0	30
31	0.109235	0.0	29.068401	33.919216	0.109198	0.0	31
32	0.108170	0.0	28.915186	33.604374	0.108230	0.0	32
33	0.107597	0.0	28.828339	33.434399	0.107614	0.0	33
34	0.107097	0.0	28.782579	33.401508	0.107208	0.0	34
35	0.107020	0.0	28.759605	33.336647	0.107089	0.0	35
36	0.107008	0.0	28.752081	33.348919	0.107058	0.0	36
37	0.106734	0.0	28.700933	33.285450	0.106874	0.0	37
38	0.106819	0.0	28.677635	33.291740	0.106894	0.0	38
39	0.107084	0.0	28.798990	33.389713	0.107378	0.0	39
40	0.106920	0.0	28.718044	33.300632	0.106926	0.0	40
41	0.106569	0.0	28.706158	33.226959	0.106627	0.0	41
42	0.106769	0.0	28.690668	33.283615	0.106660	0.0	42
43	0.106781	0.0	28.693810	33.258286	0.106792	0.0	43
44	0.106345	0.0	28.631138	33.218601	0.106401	0.0	44
45	0.106421	0.0	28.630007	33.144199	0.106280	0.0	45
46	0.106411	0.0	28.619070	33.171017	0.106372	0.0	46
47	0.106255	0.0	28.619728	33.152409	0.106287	0.0	47
48	0.106156	0.0	28.625244	33.147099	0.106215	0.0	48
49	0.106103	0.0	28.588451	33.091316	0.106074	0.0	49
50	0.105966	0.0	28.569561	33.108128	0.105920	0.0	50
51	0.105878	0.0	28.553705	33.065838	0.105861	0.0	51
52	0.105819	0.0	28.582106	33.080173	0.105948	0.0	52
53	0.105704	0.0	28.555727	33.064678	0.105822	0.0	53
54	0.105648	0.0	28.521925	33.017765	0.105616	0.0	54
55	0.105610	0.0	28.519014	32.999790	0.105482	0.0	55
56	0.105535	0.0	28.483585	32.989182	0.105510	0.0	56
57	0.105449	0.0	28.511715	32.995770	0.105506	0.0	57
58	0.105447	0.0	28.495548	32.982304	0.105376	0.0	58
59	0.105440	0.0	28.495508	32.996822	0.105406	0.0	59


```
In [27]: # Make future predictions (next 30 days, for example)
future = model.make_future_dataframe(df, periods=30 * 24, n_historic_predictions=10)
forecast = model.predict(future)
```

```
INFO - (NP.df_utils._infer_frequency) - Major frequency -15159359999997us corresponds to 0.025% of the data.
WARNING - (NP.df_utils._infer_frequency) - Dataframe has multiple frequencies. It will be resampled according to given frequency. Ignore message if actual frequency is any of the following: SM, BM, CBM, SMS, BMS, CBMS, BQ, BQS, BA, or, BAS.
INFO - (NP.df_utils.return_df_in_original_format) - Returning df with no ID column
INFO - (NP.df_utils._infer_frequency) - Major frequency h corresponds to 8.22% of the data.
WARNING - (NP.df_utils._infer_frequency) - Dataframe has multiple frequencies. It will be resampled according to given frequency. Ignore message if actual frequency is any of the following: SM, BM, CBM, SMS, BMS, CBMS, BQ, BQS, BA, or, BAS.
INFO - (NP.df_utils._infer_frequency) - Major frequency h corresponds to 8.22% of the data.
WARNING - (NP.df_utils._infer_frequency) - Dataframe has multiple frequencies. It will be resampled according to given frequency. Ignore message if actual frequency is any of the following: SM, BM, CBM, SMS, BMS, CBMS, BQ, BQS, BA, or, BAS.
INFO - (NP.data.processing._handle_missing_data) - Dropped 720 rows at the end with NaNs in 'y' column.
Predicting: |
| 0/? [00:00<...
INFO - (NP.df_utils.return_df_in_original_format) - Returning df with no ID column
```

```
In [28]: # Plot the forecast
model.plot(forecast)
```

```
ERROR - (NP.plotly.plot) - plotly-resampler is not installed. Please install it to use the resampler.
```



```
In [42]: # Using ARIMA/SARIMA with statsmodels
from statsmodels.tsa.arima.model import ARIMA
import statsmodels.api as sm
```

```
In [46]: # Assuming you have a DataFrame `df` with columns like 'Ride_Time', 'Pickup_

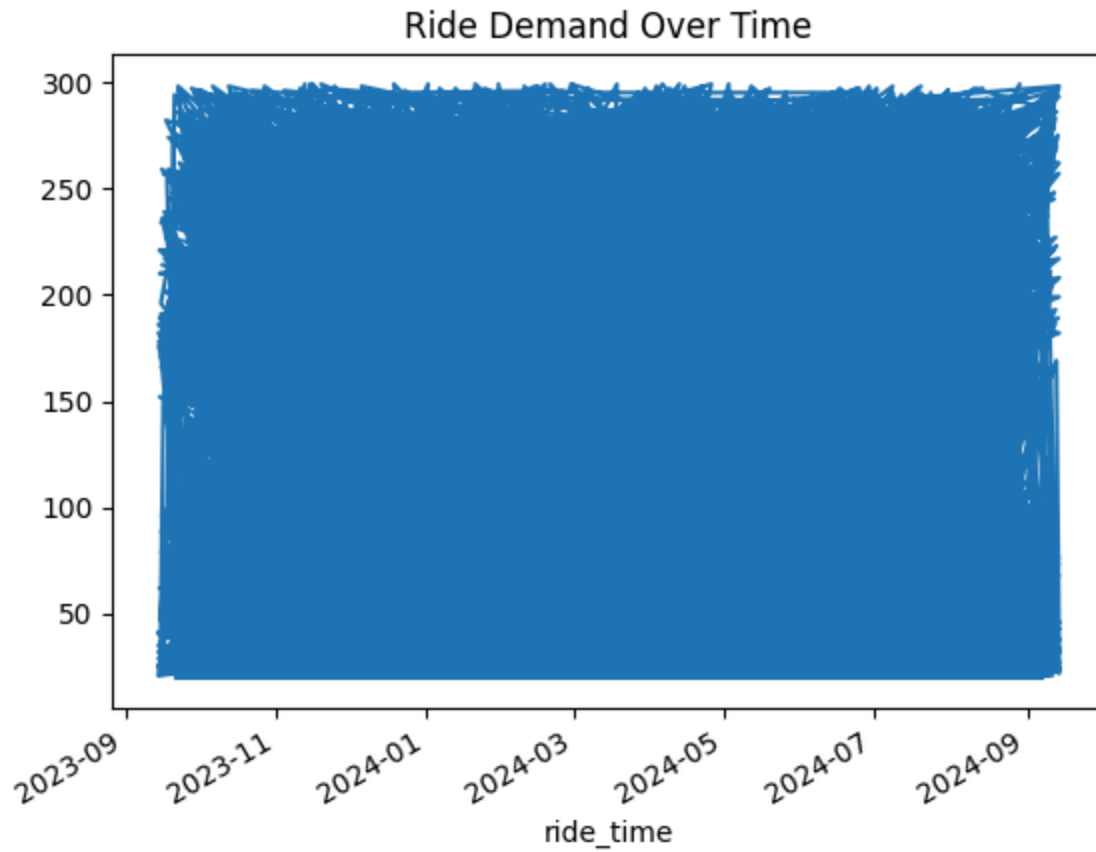
# If you have a 'Ride_Time' or similar datetime column
df['ride_time'] = pd.to_datetime(df['ride_time'])

# Generate a synthetic demand pattern based on Ride_Time
# Example: Assume higher demand during morning (7-9 AM) and evening (5-8 PM)
def generate_demand(ride_time):
    hour = ride_time.hour
    if 7 <= hour <= 9 or 17 <= hour <= 20:
        return np.random.randint(150, 300) # High demand during peak hours
    elif 10 <= hour <= 16:
        return np.random.randint(50, 100) # Lower demand during off-peak
    else:
        return np.random.randint(20, 50) # Very low demand at night

# Apply the function to create the 'demand' column
df['demand'] = df['ride_time'].apply(generate_demand)
```

```
In [47]: # Example: Prepare your ride demand time series data
# Assuming `df` has a `date` and `demand` column for ride demand over time
df['ride_time'] = pd.to_datetime(df['ride_time'])
df.set_index('ride_time', inplace=True)
```

```
In [48]: # Visualize data
df['demand'].plot()
plt.title('Ride Demand Over Time')
plt.show()
```



```
In [49]: # Split data into training and test sets
train_data = df['demand'][:int(0.8*len(df))]
test_data = df['demand'][int(0.8*len(df)):]
```

```
In [52]: # Fit the ARIMA model
model = ARIMA(train_data, order=(5, 1, 2)) # Example order
model_fit = model.fit()
```

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

```
In [53]: # model = sm.tsa.arima_model.ARIMA ## works fine now
```

```
In [55]: # model = ARIMA(data, order=(5, 1, 0))
# model_fit = model.fit()
```

```
In [57]: # Fit the ARIMA model (p, d, q parameters)
# model = ARIMA(train_data, order=(5, 1, 2)) # Example: (AR order=5, I order=1, MA order=2)
# Create an ARIMA model (replace 'order' with your desired ARIMA order)
# model = sm.tsa.arima.model.ARIMA(train_data, order=(5, 1, 2))
# arima_model = model.fit()

# Fit the ARIMA model
model = ARIMA(train_data, order=(5, 1, 2)) # Example order
arima_model = model.fit()
```

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:
```

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

```
In [58]: # Print model summary
         print(arima_model.summary())
```

SARIMAX Results

```
=====
=
Dep. Variable:          demand    No. Observations:          800
0
Model:                  ARIMA(5, 1, 2)    Log Likelihood          -46835.75
4
Date:                  Mon, 23 Dec 2024    AIC          93687.50
8
Time:                  10:17:01    BIC          93743.40
5
Sample:                0    HQIC          93706.64
1
```

```
          - 8000
Covariance Type:      opg
=====
```

```
=====
=
          coef      std err          z      P>|z|      [0.025      0.97
5]
-----
-
ar.L1      -0.9845      0.017     -58.579      0.000      -1.017      -0.95
2
ar.L2       0.0069      0.016       0.438      0.661      -0.024       0.03
8
ar.L3      -0.0020      0.016      -0.128      0.898      -0.033       0.02
9
ar.L4       0.0096      0.016       0.613      0.540      -0.021       0.04
0
ar.L5       0.0041      0.011       0.364      0.716      -0.018       0.02
6
ma.L1      -0.0012      0.022      -0.053      0.958      -0.044       0.04
2
ma.L2      -0.9988      0.023     -44.218      0.000      -1.043      -0.95
5
sigma2     7127.5898    5.23e-06    1.36e+09      0.000     7127.590     7127.59
0
=====
```

```
=====
=====
Ljung-Box (L1) (Q):          0.00    Jarque-Bera (JB):          1
339.36
Prob(Q):          0.96    Prob(JB):
0.00
Heteroskedasticity (H):      0.99    Skew:
0.96
Prob(H) (two-sided):          0.89    Kurtosis:
2.45
=====
=====
```

Warnings:

- [1] Covariance matrix calculated using the outer product of gradients (complex-step).
- [2] Covariance matrix is singular or near-singular, with condition number 1.6e+23. Standard errors may be unstable.

```
In [59]: # Forecast future values
# forecast, stderr, conf_int = arima_model.forecast(steps=len(test_data))
```

```
In [ ]: # Plot predictions against actual demand
plt.plot(test_data.index, test_data, label='Actual Demand')
plt.plot(test_data.index, forecast, color='red', label='Predicted Demand')
plt.fill_between(test_data.index, conf_int[:, 0], conf_int[:, 1], color='pink')
plt.legend()
plt.title('Actual vs Predicted Ride Demand')
plt.show()
```

```
In [62]: # Using SARIMA for Seasonality
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
In [63]: # Fit the SARIMA model with seasonal component
sarima_model = SARIMAX(train_data, order=(1, 1, 1), seasonal_order=(1, 1, 1, 1),
sarima_fit = sarima_model.fit()
```

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning:

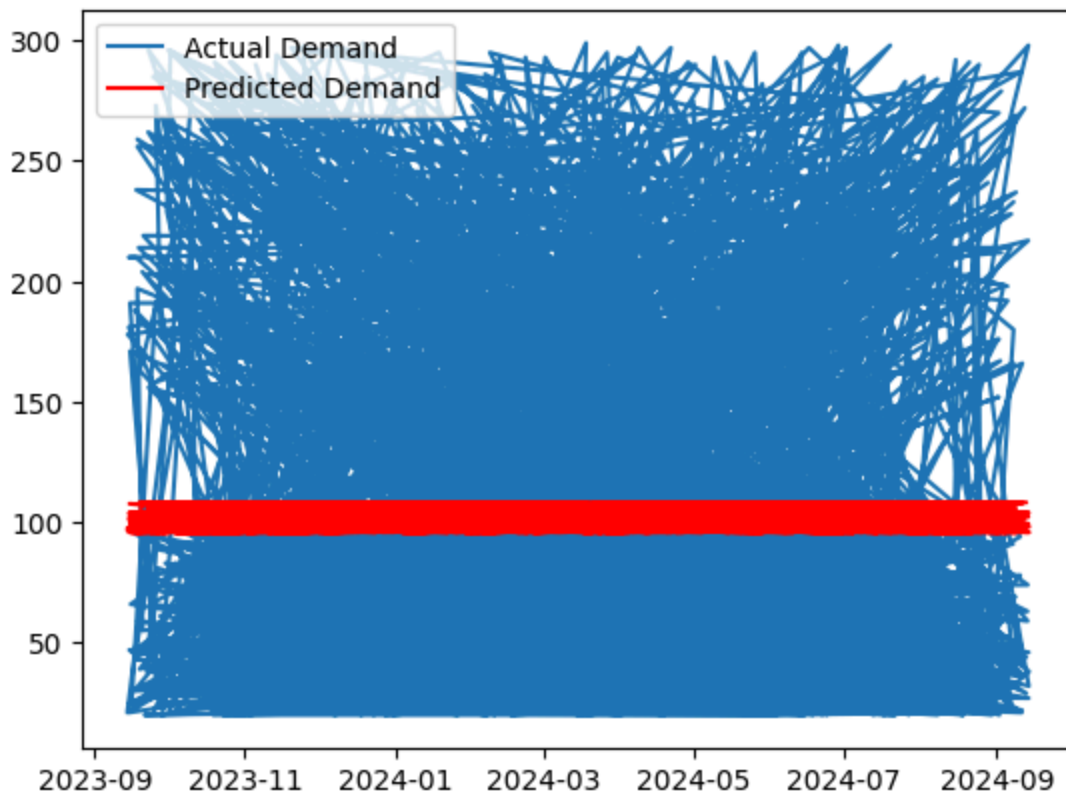
A date index has been provided, but it is not monotonic and so will be ignored when e.g. forecasting.

```
In [64]: # Forecast
forecast = sarima_fit.forecast(steps=len(test_data))
```


WARNING - (py.warnings._showwarnmsg) - C:\Users\prasad jadhav\AppData\Local\Programs\Python\Python310\lib\site-packages\statsmodels\tsa\base\tsa_model.py:836: ValueWarning:

No supported index is available. Prediction results will be given with an integer index beginning at `start`.

```
In [65]: # Plot predictions
plt.plot(test_data.index, test_data, label='Actual Demand')
plt.plot(test_data.index, forecast, color='red', label='Predicted Demand')
plt.legend()
plt.show()
```



```
In [42]: # Using LSTM (Long Short-Term Memory) with Keras/TensorFlow
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
In [47]: # Assume `df` has 'date' and 'demand' columns
# df['ride_time'] = pd.to_datetime(df['ride_time'])
# df.set_index('ride_time', inplace=True)
```

```
In [49]: # Normalize the demand data using MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
df_scaled = scaler.fit_transform(df[['demand']])
```

```
In [50]: # Create training data (use a look-back window of 60 time steps)
X_train = []
```

```
y_train = []
look_back = 60
```

```
In [51]: for i in range(look_back, len(df_scaled)):
          X_train.append(df_scaled[i-look_back:i, 0])
          y_train.append(df_scaled[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

```
In [52]: # Build LSTM model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dense(units=25))
model.add(Dense(units=1))
```

```
In [57]: # Compile and fit the model
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, batch_size=32, epochs=10) # 50
```

```
Epoch 1/10
249/249 [=====] - 32s 88ms/step - loss: 0.0917
Epoch 2/10
249/249 [=====] - 26s 103ms/step - loss: 0.0919
Epoch 3/10
249/249 [=====] - 30s 122ms/step - loss: 0.0917
Epoch 4/10
249/249 [=====] - 24s 95ms/step - loss: 0.0917
Epoch 5/10
249/249 [=====] - 25s 102ms/step - loss: 0.0917
Epoch 6/10
249/249 [=====] - 23s 93ms/step - loss: 0.0917
Epoch 7/10
249/249 [=====] - 24s 97ms/step - loss: 0.0918
Epoch 8/10
249/249 [=====] - 23s 93ms/step - loss: 0.0918
Epoch 9/10
249/249 [=====] - 22s 90ms/step - loss: 0.0918
Epoch 10/10
249/249 [=====] - 22s 89ms/step - loss: 0.0918
```

```
Out[57]: <keras.callbacks.History at 0x13bfe74c670>
```

```
In [58]: # Test data for forecasting
X_test = []
test_data = df_scaled[len(df_scaled) - len(test_data) - look_back:]

for i in range(look_back, len(test_data)):
    X_test.append(test_data[i-look_back:i, 0])

X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
```

```
In [59]: # Predictions
predictions = model.predict(X_test)
```

```
predictions = scaler.inverse_transform(predictions)
```

53/53 [=====] - 4s 36ms/step

```
In [61]: # Plot predictions
# plt.plot(df.index[-len(predictions):], test_data, label='Actual Demand')
# plt.plot(df.index[-len(predictions):], predictions, color='red', label='Pr
# plt.legend()
# plt.show()
```

```
In [ ]: '''
# Using GluonTS (DeepAR Model)
from gluonts.dataset.common import ListDataset
from gluonts.model.deepar import DeepAREstimator
from gluonts.mx.trainer import Trainer
from gluonts.evaluation import make_evaluation_predictions, Evaluator

# Step 1: Prepare your data (assuming df has 'date' and 'demand' columns)
df['date'] = pd.to_datetime(df['ride_time']) # Ensure the 'date' column is
df.set_index('date', inplace=True)

# Step 2: Create a demand column (if it's not present)
def generate_demand(ride_time):
    hour = ride_time.hour
    if 7 <= hour <= 9 or 17 <= hour <= 20:
        return np.random.randint(150, 300) # High demand during peak hours
    elif 10 <= hour <= 16:
        return np.random.randint(50, 100) # Lower demand during off-peak h
    else:
        return np.random.randint(20, 50) # Low demand during night

df['demand'] = df['ride_time'].apply(generate_demand)

# Step 3: Define your training dataset (last 30 days for test, for example)
train_data = ListDataset(['start': df.index[0], 'target': df['demand'].valu
test_data = ListDataset(['start': df.index[0], 'target': df['demand'].value

# Step 4: Define and train the DeepAR estimator
estimator = DeepAREstimator(freq="H",
                             prediction_length=30, # Forecast for the next 3
                             trainer=Trainer(epochs=10))

# Step 5: Train the model
predictor = estimator.train(training_data=train_data)

# Step 6: Make predictions using the trained model
forecast_it, ts_it = make_evaluation_predictions(dataset=test_data, predicto

# Convert the forecast object to pandas DataFrame for easier visualization
forecasts = list(forecast_it)
targets = list(ts_it)

# Step 7: Visualize the predictions vs actual values
for target, forecast in zip(targets, forecasts):
    target.plot(label='actual')
    forecast.plot(label='forecast', color='red')
```

```

plt.legend()
plt.show()

# Step 8: Evaluate the model performance
evaluator = Evaluator()
agg_metrics, item_metrics = evaluator(iter(targets), iter(forecasts))
print(f"CRPS: {agg_metrics['CRPS']}")
'''

```

Driver Performance Prediction:

- **Problem** : Predict driver performance based on ratings and ride attributes.
- **Approach** : Classification using features like driver_id, ride_distance, fare_amount, and driver_rating.
- **Model** : Random Forest, XGBoost, or SVM.

```

In [70]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

```

```

In [113... # Classify driver performance as good (>=4) or bad (<4)
df['good_performance'] = (df['driver_rating'] >= 4).astype(int)

```

```

In [114... # Feature selection
X = df[['ride_distance', 'ride_duration', 'fare_amount', 'battery_level', 'c
y = df['good_performance']

```

```

In [115... # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

```

```

In [72]: # Model training
model = RandomForestClassifier()
model.fit(X_train, y_train)

```

```

Out[72]: RandomForestClassifier ⓘ ?
RandomForestClassifier()

```

```

In [73]: # Predictions and evaluation
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")

```

Accuracy: 0.701120797011208

Vehicle Maintenance Prediction:

- **Problem** : Predict when a vehicle will require maintenance.
- **Approach** : Classification using features like vehicle_model, battery_level, ride_distance, and vehicle_maintenance_flag.
- **Model** : Logistic Regression, Decision Trees, or Gradient Boosting.

```
In [117... from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report
```

```
In [118... # Feature selection
X = df[['ride_distance', 'ride_duration', 'fare_amount', 'battery_level']]
y = df['vehicle_maintenance_flag']
```

```
In [120... # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
In [121... # Model training
model = GradientBoostingClassifier()
model.fit(X_train, y_train)
```

```
Out[121... ▾ GradientBoostingClassifier ⓘ ⓘ
GradientBoostingClassifier()
```

```
In [122... # Predictions and evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
False	0.95	1.00	0.97	1522
True	0.00	0.00	0.00	84
accuracy			0.95	1606
macro avg	0.47	0.50	0.49	1606
weighted avg	0.90	0.95	0.92	1606

Ride Cancellation Prediction:

- **Problem**: Predict if a scheduled ride will be canceled.
- **Approach**: Classification using features like schedule_time, ride_distance, traffic_conditions, and cancellation_flag.
- **Model**: Logistic Regression, K-Nearest Neighbors, or Neural Networks.

```
In [124... from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
```

```
In [125... setattr(pandas, 'Int64Index', pandas.Index)
setattr(pandas, 'Float64Index', pandas.Index)
```

```
In [126... # Feature selection
X = df[['ride_distance', 'ride_duration', 'traffic_conditions', 'weather_con
y = df['cancellation_flag']
```

```
In [130... # Encode categorical variables
X = pd.get_dummies(X, drop_first=True)
```

```
In [128... # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
In [98]: # Model training
model = XGBClassifier()
model.fit(X_train, y_train)
```

[09:00:05] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

```
Out[98]: XGBClassifier
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
              importance_type='gain', interaction_constraints='',
              learning_rate=0.300000012, max_delta_step=0, max_depth=6,
              min_child_weight=1, missing=nan, monotone_constraints='()',
```

```
In [99]: # Predictions and evaluation
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

Accuracy: 0.8991282689912827

Fare Prediction:

- **Problem**: Predict the fare amount for a given ride.
- **Approach**: Regression using features like ride_distance, ride_duration, traffic_conditions, and weather_conditions.
- **Model**: Linear Regression, XGBoost, or Neural Networks.

```
In [100... from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
In [101... # Feature selection
X = df[['ride_distance', 'ride_duration', 'traffic_conditions', 'weather_con
y = df['fare_amount']
```

```
In [102... # Encode categorical variables
X = pd.get_dummies(X, drop_first=True)
```

```
In [103... # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
In [104... # Model training
model = LinearRegression()
model.fit(X_train, y_train)
```

```
Out[104... LinearRegression ⓘ ?
LinearRegression()
```

```
In [105... # Predictions and evaluation
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
```

Mean Squared Error: 8.502901404482912e-06

Battery Level Prediction:

- **Problem**: Predict the remaining battery level of a vehicle after a ride.
- **Approach**: Regression using features like ride_distance, ride_duration, vehicle_model, and battery_level.
- **Model**: Linear Regression, Decision Trees, or Random Forest.

```
In [106... from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error
```

```
In [107... # Feature selection
X = df[['ride_distance', 'ride_duration', 'vehicle_model']]
y = df['battery_level']
```

```
In [108... # Encode categorical variables
X = pd.get_dummies(X, drop_first=True)
```

```
In [109... # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
In [110... # Model training
model = DecisionTreeRegressor()
model.fit(X_train, y_train)
```

```
Out[110... DecisionTreeRegressor ⓘ ?
DecisionTreeRegressor()
```

```
In [111... # Predictions and evaluation
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
print(f"Mean Absolute Error: {mae}")
```

Mean Absolute Error: 27.152132627646324

Advance Code Example

```
In [ ]: # Ride Demand Forecasting (Time Series Forecasting)

'''
# Adding external regressors to Prophet
prophet_df['weather_conditions'] = pd.get_dummies(df['weather_conditions'],

model.add_regressor('weather_conditions')
model.fit(prophet_df)
'''
```

```
In [116... # Driver Performance Prediction (Classification)

from sklearn.model_selection import cross_val_score

model = RandomForestClassifier()
scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
print(f'Cross-Validation Accuracy: {np.mean(scores)}')
```

Cross-Validation Accuracy: 0.7149979999272701

```
In [123... # Vehicle Maintenance Prediction (Classification)

from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)

# Train with balanced data
model.fit(X_res, y_res)

# Model training
model = GradientBoostingClassifier()
model.fit(X_train, y_train)

# Predictions and evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
False	0.95	1.00	0.97	1522
True	0.00	0.00	0.00	84
accuracy			0.95	1606
macro avg	0.47	0.50	0.49	1606
weighted avg	0.90	0.95	0.92	1606

```
In [131... # Ride Cancellation Prediction (Classification)

from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV
```



```
# Hyperparameter tuning for XGBoost
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7]
}

xgb = XGBClassifier()
grid_search = RandomizedSearchCV(xgb, param_distributions=param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Evaluate best model
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Predictions and evaluation
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

[09:14:23] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:24] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:24] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:25] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:26] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:27] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:27] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:28] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:28] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:29] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:29] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:30] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.

4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:30] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:31] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:31] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:32] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:32] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:33] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:33] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:34] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:34] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:35] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:36] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation me

tric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:36] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:37] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:37] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:38] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:39] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:40] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:40] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:41] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:42] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:44] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:45] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:47] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:49] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:49] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:50] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:51] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.

tric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

[09:14:57] WARNING: C:/Users/Administrator/workspace/xgboost-win64_release_1.4.0/src/learner.cc:1095: Starting in XGBoost 1.3.0, the default evaluation metric used with the objective 'binary:logistic' was changed from 'error' to 'logloss'. Explicitly set eval_metric if you'd like to restore the old behavior.

ValueError

Traceback (most recent call last)

Cell In[131], line 22

```
19 y_pred = best_model.predict(X_test)
21 # Predictions and evaluation
--> 22 y_pred = model.predict(X_test)
23 print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\ensemble_gb.py:1611, in GradientBoostingClassifier.predict(self, X)

```
1596 def predict(self, X):
1597     """Predict class for X.
1598
1599     Parameters
1600     (...)
1601         The predicted values.
1602     """
-> 1611 raw_predictions = self.decision_function(X)
1612 if raw_predictions.ndim == 1: # decision_function already squeezed it
1613     encoded_classes = (raw_predictions >= 0).astype(int)
```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\ensemble_gb.py:1564, in GradientBoostingClassifier.decision_function(self, X)

```
1545 def decision_function(self, X):
1546     """Compute the decision function of ``X``.
1547
1548     Parameters
1549     (...)
1550         array of shape (n_samples,).
1551     """
-> 1564 X = self._validate_data(
1565     X, dtype=DTYPE, order="C", accept_sparse="csr", reset=False
1566 )
1567 raw_predictions = self._raw_predict(X)
1568 if raw_predictions.shape[1] == 1:
```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\base.py:608, in BaseEstimator._validate_data(self, X, y, reset, validate_separately, cast_to_ndarray, **check_params)

```
537 def _validate_data(
538     self,
539     X="no_validation",
540     (...)
541     **check_params,
542 ):
543     """Validate input data and set or check the `n_features_in` attribute.
544
545     Parameters
546     (...)
547         validated.
548     """
--> 608 self._check_feature_names(X, reset=reset)
609 if y is None and self._get_tags()["requires_y"]:
610     raise ValueError(
```



```

612         f"This {self.__class__.__name__} estimator "
613         "requires y to be passed, but the target y is None."
614     )

```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\base.py:535, in BaseEstimator._check_feature_names(self, X, reset)

```

530 if not missing_names and not unexpected_names:
531     message += (
532         "Feature names must be in the same order as they were in fi
t.\n"
533     )
--> 535 raise ValueError(message)

```

ValueError: The feature names should match those that were passed during fit. Feature names unseen at fit time:

- traffic_conditions_light
- traffic_conditions_moderate
- weather_conditions_rainy
- weather_conditions_sunny

Feature names seen at fit time, yet now missing:

- battery_level
- fare_amount

```

In [ ]: # Fare Prediction (Regression)

from xgboost import XGBRegressor

model = XGBRegressor(n_estimators=500, learning_rate=0.05)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

```

```

In [ ]: # Battery Level Prediction (Regression)

# Additional features such as traffic and weather
X = df[['ride_distance', 'ride_duration', 'vehicle_model', 'traffic_conditio
X = pd.get_dummies(X, drop_first=True)

# Use RandomForest or XGBoost for regression
model = RandomForestRegressor(n_estimators=100)
model.fit(X_train, y_train)

```

More Experiment

Feature Preprocessing

```

In [199... # Define mapping dictionaries
pickup_location_mapping = {
    "Chennai": 1,
    "Hyderabad": 2,
    "Kolkata": 3,
    "Jaipur": 4,
    "Mumbai": 5,

```

```

    "Pune": 6,
    "Bangalore": 7,
    "Delhi": 8
}

dropoff_location_mapping = {
    "Chennai": 1,
    "Bangalore": 2,
    "Kolkata": 3,
    "Jaipur": 4,
    "Pune": 5,
    "Hyderabad": 6,
    "Delhi": 7,
    "Mumbai": 8
}

# Function to apply mapping
def apply_pickup_location_mapping(df, column_name):
    df[column_name] = df[column_name].map(pickup_location_mapping)
    return df

def apply_dropoff_location_mapping(df, column_name):
    df[column_name] = df[column_name].map(dropoff_location_mapping)
    return df

```

```

In [200... # Apply mappings
df = apply_pickup_location_mapping(df, "pickup_location")
df = apply_dropoff_location_mapping(df, "dropoff_location")

```

```

In [201... # Define mapping dictionaries
payment_method_mapping = {
    "credit": 1,
    "wallet": 2,
    "debit": 3,
    "cash": 4,
    "UPI": 5
}

vehicle_model_mapping = {
    "MG ZS Electric": 1,
    "Tata e-Tigor": 2,
    "Mahindra e-Verito": 3,
    "Hyundai Kona Electric": 4,
    "Tata Xpres-T EV": 5
}

traffic_conditions_mapping = {
    "light": 1,
    "heavy": 2,
    "moderate": 3
}

weather_conditions_mapping = {
    "sunny": 1,
    "cloudy": 2,

```

```

    "rainy": 3
}

# Function to apply mapping
def apply_mapping(df, column_name, mapping_dict):
    df[column_name] = df[column_name].map(mapping_dict)
    return df

```

```

In [202]: # Apply mappings
df = apply_mapping(df, "payment_method", payment_method_mapping)
df = apply_mapping(df, "vehicle_model", vehicle_model_mapping)
df = apply_mapping(df, "traffic_conditions", traffic_conditions_mapping)
df = apply_mapping(df, "weather_conditions", weather_conditions_mapping)

```

Driver Performance Prediction (Classification)

- This model predicts driver performance (good or bad rating) based on ride attributes.

```

In [66]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

```

```

In [72]: df['driver_rating'].describe()

```

```

Out[72]: count    10000.00000
mean         3.02588
std          1.15403
min           1.00000
25%           2.00000
50%           3.00000
75%           4.00000
max           5.00000
Name: driver_rating, dtype: float64

```

```

In [71]: # Classify driver performance as good (>=4) or bad (<4)
df['driver_performance'] = (df['driver_rating'] >= 4).astype(int)

```

```

In [75]: df['driver_performance'].value_counts()

```

```

Out[75]: driver_performance
0      7263
1      2737
Name: count, dtype: int64

```

```

In [95]: # Feature selection
X = df[['ride_distance', 'ride_duration', 'fare_amount', 'battery_level', 'c
y = df['driver_performance']

```

```

In [96]: # Imbalanced Data Handling
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_res,y_res = smote.fit_resample(X,y)

```

```
In [97]: X_res,y_res = SMOTE().fit_resample(X,y)
```

```
In [98]: from sklearn.model_selection import train_test_split

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=
```

```
In [99]: # from sklearn.preprocessing import StandardScaler

# scaler = StandardScaler()
# X_train = scaler.fit_transform(X_train)
# X_test = scaler.fit_transform(X_test)
```

```
In [100... # Model training
r_model = RandomForestClassifier()
r_model.fit(X_train, y_train)
```

```
Out[100... RandomForestClassifier ⓘ ?
RandomForestClassifier()
```

```
In [101... # Predictions and evaluation
y_pred = r_model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

Accuracy: 0.7040605643496215

```
In [103... from sklearn.metrics import accuracy_score, classification_report

def get_best_model_report(models, X_train, y_train, X_test, y_test):
    """
    Train models, evaluate their accuracy, and display the classification re

    Args:
    models (dict): A dictionary with model names as keys and model objects a
    X_train, y_train: Training features and labels.
    X_test, y_test: Testing features and labels.
    """
    best_model_name = None
    best_model = None
    best_accuracy = 0
    y_best_pred = None

    # Train, predict, and evaluate each model
    for model_name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        print(f'Accuracy {model_name}: {accuracy:.4f}')

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_model_name = model_name
            best_model = model
```

```

        y_best_pred = y_pred

    print(f"\nBest Model: {best_model_name} with Accuracy: {best_accuracy:.4}")
    print("Classification Report for Best Model:\n")
    print(classification_report(y_test, y_best_pred))

# Example usage
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Define models
models = {
    "LogisticRegression": LogisticRegression(),
    "KNeighborsClassifier": KNeighborsClassifier(),
    "SVC": SVC(),
    "DecisionTreeClassifier": DecisionTreeClassifier(),
    "RandomForestClassifier": RandomForestClassifier(),
    "GradientBoostingClassifier": GradientBoostingClassifier()
}

```

In [104... *# Call the function*

```

get_best_model_report(models, X_train, y_train, X_test, y_test)

```

```

Accuracy LogisticRegression: 0.5251
Accuracy KNeighborsClassifier: 0.6590
Accuracy SVC: 0.5165
Accuracy DecisionTreeClassifier: 0.6266
Accuracy RandomForestClassifier: 0.7058
Accuracy GradientBoostingClassifier: 0.5609

```

Best Model: RandomForestClassifier with Accuracy: 0.7058

Classification Report for Best Model:

	precision	recall	f1-score	support
0	0.72	0.68	0.70	1471
1	0.69	0.73	0.71	1435
accuracy			0.71	2906
macro avg	0.71	0.71	0.71	2906
weighted avg	0.71	0.71	0.71	2906

In [114... *# Assuming `model` is already trained and loaded*

```

print("Driver Performance Prediction")

# Input data
ride_distance = float(input("Enter the Ride Distances: "))
ride_duration = float(input("Enter the Ride Durations: "))
fare_amount = float(input("Enter the Fare Smount: "))
battery_level = float(input("Enter the Battery Level: "))

```

```

charging_station_proximity = float(input("Enter the Charging Station Proximi

# Create a feature array
input_point = np.array([[ride_distance, ride_duration, fare_amount, battery_

# Make a prediction
prediction = r_model.predict(input_point)

# Check the prediction and print the result
if prediction[0] == 1:
    print("Performance is Good")
else: # No need to check for `prediction[0] == 0` since it's implied
    print("Performance is Bad")

```

Driver Performance Prediction
Enter the Ride Distances: 20.933662
Enter the Ride Durations: 46
Enter the Fare Smount: 416.516421
Enter the Battery Level: 64.795554
Enter the Charging Station Proximity: 3.231813
Performance is Good

Battery Level Prediction (Regression)

- This model predicts the battery level after a ride based on distance and other features.

```

In [119... from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

```

```

In [121... # Feature selection
X = df[['ride_distance', 'ride_duration', 'vehicle_model']]
y = df['battery_level']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

# Model training
b_model = RandomForestRegressor()
b_model.fit(X_train, y_train)

# Predictions and evaluation
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
print(f"Mean Absolute Error: {mae}")

```

Mean Absolute Error: 21.14857442488095

```

In [127... print("Battery Level Prediction")

# Input data
ride_distance = float(input("Enter the Ride Distance: "))
ride_duration = int(input("Enter the Ride Duration: "))
vehicle_model = int(input("Enter the Vehicle Model: "))

```

```

# Create a feature array
input_point = np.array([[ride_distance, ride_duration, vehicle_model]])
# Make a prediction
prediction = b_model.predict(input_point)
print(f'Battery: {prediction[0]} Level Prediction')

```

Battery Level Prediction
Enter the Ride Distance: 41.85
Enter the Ride Duration: 102
Enter the Vehicle Model: 3
Battery: 30.840899999999962 Level Prediction

Fare Prediction (Regression)

- This model predicts the fare of a ride based on distance, duration, and other factors.

```

In [130... from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

```

```

In [131... # Feature selection
X = df[['ride_distance', 'ride_duration', 'traffic_conditions', 'weather_con
y = df['fare_amount']

# Encode categorical variables
X = pd.get_dummies(X, drop_first=True)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

# Model training
a_model = LinearRegression()
a_model.fit(X_train, y_train)

# Predictions and evaluation
y_pred = a_model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

```

Mean Squared Error: 8.276360781722184e-06

```

In [136... # Input data
ride_distance = float(input("Enter the Ride Distance: "))
ride_duration = int(input("Enter the Ride Duration: "))
traffic_condition = int(input("Enter the Traffic Condition: "))
weather_condition = int(input("Enter the Weather Condition: "))

# Create a feature array
input_point = np.array([[ride_distance, ride_duration, traffic_condition, we
# Make a prediction
prediction = a_model.predict(input_point)
print(f'Your: {prediction[0]} Fare Amount')

```

Enter the Ride Distance: 22.98
Enter the Ride Duration: 115
Enter the Traffic Condition: 2
Enter the Weather Condition: 3
Your: 457.23305322018587 Fare Amount

Ride Cancellation Prediction (Classification)

- This model predicts if a scheduled ride will be canceled based on different ride factors.

```
In [203... from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

In [204... # Convert all values to strings and handle NaN
df['cancellation_flag'] = df['cancellation_flag'].astype(str).str.strip().st

# Apply mapping
df['cancellation_flag'] = df['cancellation_flag'].map({'True': 0, 'False': 1

In [205... df['cancellation_flag'].value_counts()

Out[205... cancellation_flag
1      9018
0       982
Name: count, dtype: int64

In [206... # Feature selection
X = df[['vehicle_model', 'ride_distance', 'ride_duration', 'traffic_conditio
y = df['cancellation_flag']

In [211... from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE, ADASYN, RandomOverSampler, Border1
import pandas as pd

def resample_and_split_data(X, y, method="SMOTE", test_size=0.2, random_stat
"""
    Split data into train and test sets, then resample the training data to

    Parameters:
        X (pd.DataFrame or np.ndarray): Feature matrix.
        y (pd.Series or np.ndarray): Target variable.
        method (str): Sampling method. Options: "SMOTE", "ADASYN", "RandomOv
        test_size (float): Proportion of the dataset to include in the test
        random_state (int): Random state for reproducibility.
        **kwargs: Additional arguments for the sampler.

    Returns:
        X_train_res (pd.DataFrame or np.ndarray): Resampled training feature
        X_test (pd.DataFrame or np.ndarray): Test feature matrix.
        y_train_res (pd.Series or np.ndarray): Resampled training target var
        y_test (pd.Series or np.ndarray): Test target variable.
"""
    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(
```



```

        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Select the sampling method
    sampler = None
    if method == "SMOTE":
        sampler = SMOTE(random_state=random_state, **kwargs)
    elif method == "ADASYN":
        sampler = ADASYN(random_state=random_state, **kwargs)
    elif method == "RandomOverSampler":
        sampler = RandomOverSampler(random_state=random_state, **kwargs)
    elif method == "BorderlineSMOTE":
        sampler = BorderlineSMOTE(random_state=random_state, **kwargs)
    else:
        raise ValueError(f"Unknown method: {method}. Choose from 'SMOTE', 'A

    # Resample the training data
    X_train_res, y_train_res = sampler.fit_resample(X_train, y_train)

    return X_train_res, X_test, y_train_res, y_test

# Example usage
if __name__ == "__main__":
    # Replace X and y with your actual dataset
    X_train_res, X_test, y_train_res, y_test = resample_and_split_data(X, y,

    # Print class distributions
    print("Training Class Distribution After Resampling:")
    print(pd.Series(y_train_res).value_counts())
    print("\nTesting Class Distribution:")
    print(pd.Series(y_test).value_counts())

```

Training Class Distribution After Resampling:

cancellation_flag

1 7214

0 7214

Name: count, dtype: int64

Testing Class Distribution:

cancellation_flag

1 1804

0 196

Name: count, dtype: int64

```

In [215... # Split the data
# X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_siz

# Model training
c_model = XGBClassifier()
c_model.fit(X_train_res, y_train_res)

# Predictions and evaluation
y_pred = c_model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")

```

Accuracy: 0.719

```
In [216... # Model training
c_model = RandomForestClassifier()
c_model.fit(X_train_res, y_train_res)

# Predictions and evaluation
y_pred = c_model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
```

Accuracy: 0.752

```
In [224... print("Ride Cancellation Prediction")

# Input data
vehicle_model = int(input("Enter the Vehicle Model: "))
ride_distance = float(input("Enter the Ride Distance: "))
ride_duration = int(input("Enter the Ride Duration: "))
traffic_condition = int(input("Enter the Traffic Condition: "))
weather_condition = int(input("Enter the Weather Condition: "))

# Create a feature array
input_point = np.array([vehicle_model, ride_distance, ride_duration, traffi

# Make a prediction
prediction = c_model.predict(input_point)

# Display the result based on the prediction
if prediction[0] == 1:
    print("Your Ride is Cancelled.")
elif prediction[0] == 0:
    print("Your Ride is On the Way.")
```

Ride Cancellation Prediction
Enter the Vehicle Model: 2
Enter the Ride Distance: 26.214885
Enter the Ride Duration: 14
Enter the Traffic Condition: 2
Enter the Weather Condition: 2
Your Ride is On the Way.

```
In [ ]: """
Traffic and weather conditions often necessitate ride cancellations. However
high cancellation fees discourage individual riders from canceling,
leading to a range of problems for both riders and drivers."
"""
```

```
In [225... # More Advance Working Sonn..!
# Notebook Project By : PRASAD JADHAV (ML-ENG)
# LinkedIn: linkedin.com/in/prasadmjadhav2 | Github: github.com/prasadmjadha
```

Thank You!